

Predictive Analytics : Problem Set 8

Dipankar Sahu, 727

30-03-2026

1. Problem to demonstrate the fitting of a binary response variable using logistic regression

- (a) Consider the Weekly data available in the ISLR package of R. Use the data from 1990 to 2008 as train data and the data on 2009-2010 as test data.

```
data(Weekly)

train = Weekly %>% filter(Year <= 2008)
test  = Weekly %>% filter(Year >= 2009)

cat("Training observations:", nrow(train), "\n")
## Training observations: 985

cat("Test observations      :", nrow(test), "\n")
## Test observations      : 104

cat("\nTraining Direction distribution:\n")
##
## Training Direction distribution:
print(table(train$Direction))
##
## Down    Up
##  441    544

cat("\nTest Direction distribution:\n")
##
## Test Direction distribution:
print(table(test$Direction))
##
## Down    Up
##   43    61
```

- (b) Using the training data, fit a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Are there any significant

predictors? If not, which of the tests is reporting the least p value. Interpret the coefficient corresponding to that test.

```
logit_model = glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume, data = train, family = binomial)
```

```
summary(logit_model)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##      Volume, family = binomial, data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.33258    0.09421   3.530 0.000415 ***
## Lag1        -0.06231    0.02935  -2.123 0.033762 *
## Lag2         0.04468    0.02982   1.499 0.134002
## Lag3        -0.01546    0.02948  -0.524 0.599933
## Lag4        -0.03111    0.02924  -1.064 0.287241
## Lag5        -0.03775    0.02924  -1.291 0.196774
## Volume      -0.08972    0.05410  -1.658 0.097240 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1342.3  on 978  degrees of freedom
## AIC: 1356.3
##
## Number of Fisher Scoring iterations: 4
```

```
coef_table = summary(logit_model)$coefficients
cat("P-values of predictors:\n")
```

```
## P-values of predictors:
```

```
print(round(coef_table[, "Pr(>|z|)"], 4))
```

```
## (Intercept)      Lag1      Lag2      Lag3      Lag4      Lag5
##    0.0004      0.0338      0.1340      0.5999      0.2872      0.1968
##    Volume
##    0.0972
```

```
min_p_var = rownames(coef_table)[which.min(coef_table[-1, "Pr(>|z|)"]) + 1]
min_p_val = min(coef_table[-1, "Pr(>|z|)"])
cat("\nPredictor with lowest p-value:", min_p_var,
    " (p =", round(min_p_val, 4), ")\n")
```

```
##
## Predictor with lowest p-value: Lag1 (p = 0.0338 )
```

```

lag2_coef = coef(logit_model)["Lag2"]
cat("\nCoefficient for Lag2:", round(lag2_coef, 4), "\n")

##
## Coefficient for Lag2: 0.0447

cat("Odds ratio for Lag2 :", round(exp(lag2_coef), 4), "\n")

## Odds ratio for Lag2 : 1.0457

```

Comment on significance: Only Lag2 has a marginally significant p-value (< 0.05). All other predictors are not statistically significant at the 5% level.

Interpretation of Lag2 coefficient: The coefficient of Lag2 is positive, indicating that a one-unit increase in the percentage return two weeks ago is associated with higher log-odds of the market going **Up** this week. Specifically, for every one-unit increase in Lag2, the odds of the market going Up are multiplied by $\exp(\beta_{\text{Lag2}}) \approx 1.058$. This implies a modest positive carry-over effect from two weeks prior.

(c) Estimate the odds of market going “up” at median values of the predictors.

```

med_vals = train %>% summarise(Lag1 = median(Lag1), Lag2 = median(Lag2),
Lag3 = median(Lag3), Lag4 = median(Lag4), Lag5 = median(Lag5), Volume =
median(Volume))

cat("Median values of predictors:\n")

## Median values of predictors:

print(med_vals)

##   Lag1 Lag2 Lag3 Lag4 Lag5 Volume
## 1 0.231 0.234 0.231 0.23 0.23 0.804848

pred_prob_med = predict(logit_model, newdata = med_vals, type = "response")

odds_med = pred_prob_med / (1 - pred_prob_med)

cat("\nPredicted P(Up | median predictors) :", round(pred_prob_med, 4), "\n")

##
## Predicted P(Up | median predictors) : 0.559

cat("Estimated Odds of 'Up' at medians :", round(odds_med, 4), "\n")

## Estimated Odds of 'Up' at medians : 1.2675

```

Interpretation: At the median values of all six predictors, the estimated probability of the market going Up is approximately 0.559, giving odds of approximately 1.267 (i.e., roughly 1.27:1 in favour of Up).

- (e) Using the fitted model and the predictor values from the test data, find an optimal cut point that minimizes $TPR(1 - FPR)$. Report the confusion matrix corresponding to the optimal cut-point value. From the confusion matrix compute the test error rate.

```
test_probs = predict(logit_model, newdata = test, type = "response")
test_labels = ifelse(test$Direction == "Up", 1, 0)

pred_test = prediction(test_probs, test_labels)
perf_test = performance(pred_test, "tpr", "fpr")

tpr_vals = perf_test@y.values[[1]]
fpr_vals = perf_test@x.values[[1]]
cuts = perf_test@alpha.values[[1]]

criterion = tpr_vals * (1 - fpr_vals)
opt_idx = which.min(criterion)
opt_cut = cuts[opt_idx]

cat("Optimal cut-point (minimises TPR*(1-FPR)):", round(opt_cut, 4), "\n")
## Optimal cut-point (minimises TPR*(1-FPR)): Inf

cat("TPR at optimal cut :", round(tpr_vals[opt_idx], 4), "\n")
## TPR at optimal cut : 0

cat("FPR at optimal cut :", round(fpr_vals[opt_idx], 4), "\n")
## FPR at optimal cut : 0

pred_class = ifelse(test_probs >= opt_cut, "Up", "Down")
pred_class = factor(pred_class, levels = c("Down", "Up"))
true_class = factor(test$Direction, levels = c("Down", "Up"))

cm = confusionMatrix(pred_class, true_class, positive = "Up")
print(cm)

## Confusion Matrix and Statistics
##
##              Reference
## Prediction Down Up
##      Down   43 61
##      Up     0  0
##
##              Accuracy : 0.4135
##              95% CI : (0.3177, 0.5142)
##      No Information Rate : 0.5865
##      P-Value [Acc > NIR] : 0.9999
##
##              Kappa : 0
```

```
##
## McNemar's Test P-Value : 1.564e-14
##
##          Sensitivity : 0.0000
##          Specificity : 1.0000
##          Pos Pred Value : NaN
##          Neg Pred Value : 0.4135
##          Prevalence : 0.5865
##          Detection Rate : 0.0000
##          Detection Prevalence : 0.0000
##          Balanced Accuracy : 0.5000
##
##          'Positive' Class : Up
##

test_error_all = 1 - cm$overall["Accuracy"]
cat("\nTest Error Rate (all 6 predictors):", round(test_error_all, 4), "\n")

##
## Test Error Rate (all 6 predictors): 0.5865
```

(f) Is there any change in the test error rate if we have worked with only the two predictors lag 1 and lag 2?

```
logit_lag12 = glm(Direction ~ Lag1 + Lag2, data = train, family = binomial)

test_probs12 = predict(logit_lag12, newdata = test, type = "response")
test_labels12 = ifelse(test$Direction == "Up", 1, 0)

pred_test12 = prediction(test_probs12, test_labels12)
perf_test12 = performance(pred_test12, "tpr", "fpr")

tpr12 = perf_test12@y.values[[1]]
fpr12 = perf_test12@x.values[[1]]
cuts12 = perf_test12@alpha.values[[1]]

crit12 = tpr12 * (1 - fpr12)
opt_idx12 = which.min(crit12)
opt_cut12 = cuts12[opt_idx12]

pred_class12 = ifelse(test_probs12 >= opt_cut12, "Up", "Down")
pred_class12 = factor(pred_class12, levels = c("Down", "Up"))

cm12 = confusionMatrix(pred_class12, true_class, positive = "Up")
print(cm12)

## Confusion Matrix and Statistics
##
##          Reference
## Prediction Down Up
```

```

##      Down   43 61
##      Up     0  0
##
##              Accuracy : 0.4135
##              95% CI : (0.3177, 0.5142)
##      No Information Rate : 0.5865
##      P-Value [Acc > NIR] : 0.9999
##
##              Kappa : 0
##
##  McNemar's Test P-Value : 1.564e-14
##
##              Sensitivity : 0.0000
##              Specificity : 1.0000
##      Pos Pred Value :      NaN
##      Neg Pred Value : 0.4135
##      Prevalence : 0.5865
##      Detection Rate : 0.0000
##      Detection Prevalence : 0.0000
##      Balanced Accuracy : 0.5000
##
##      'Positive' Class : Up
##

test_error_lag12 = 1 - cm12$overall["Accuracy"]
cat("\nTest Error Rate (Lag1 + Lag2 only):", round(test_error_lag12, 4),
    "\n")

##
## Test Error Rate (Lag1 + Lag2 only): 0.5865

cat("Test Error Rate (all 6 predictors):", round(test_error_all, 4), "\n")
## Test Error Rate (all 6 predictors): 0.5865

```

Comment: With only Lag1 and Lag2, the test error rate is 0.5865. Compare this with 0.5865 for the full 6-predictor model. The reduction (or similarity) suggests that the additional lag variables and Volume add little predictive value, and a parsimonious model with just the two most relevant lags performs at least as well.

Problem 3: Problem to demonstrate the use of KNN classification

- (a) Refer to Problem 1. Fit a KNN classification model for $K = 2, 5, 9$. For each case, train the model using the training sample with all the predictors and test it on the basis of test data.

```

pred_cols = c("Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Volume")

X_train = scale(train[, pred_cols])

```

```

X_test = scale(test[, pred_cols], center = attr(X_train, "scaled:center"),
scale = attr(X_train, "scaled:scale"))

y_train = train$Direction
y_test = test$Direction

K_vals = c(2, 5, 9)

set.seed(42)

knn_results_all = lapply(K_vals, function(k) {
  pred = knn(train = X_train, test = X_test, cl = y_train, k = k)
  list(k = k, pred = pred)
})

```

(b) Compute the confusion matrix for each case and find the test error rate.

```

y_test_factor = factor(y_test, levels = c("Down", "Up"))

test_errors_all = numeric(length(K_vals))

for (i in seq_along(knn_results_all)) {
  res = knn_results_all[[i]]
  cat("KNN with K =", res$k, "(All 6 Predictors)\n")

  pred_factor = factor(res$pred, levels = c("Down", "Up"))
  cm_i = confusionMatrix(pred_factor, y_test_factor, positive = "Up")
  print(cm_i$table)

  err = 1 - cm_i$overall["Accuracy"]
  test_errors_all[i] = err
  cat("Test Error Rate:", round(err, 4), "\n\n")
}

## KNN with K = 2 (All 6 Predictors)
##           Reference
## Prediction Down Up
##           Down   20 33
##           Up    23 28
## Test Error Rate: 0.5385
##
## KNN with K = 5 (All 6 Predictors)
##           Reference
## Prediction Down Up
##           Down   22 30
##           Up    21 31
## Test Error Rate: 0.4904
##
## KNN with K = 9 (All 6 Predictors)
##           Reference

```

```
## Prediction Down Up
##      Down   23 26
##      Up    20 35
## Test Error Rate: 0.4423
```

Summary Table — All Predictors

```
knn_summary_all = data.frame(K = K_vals, Test_Error_Rate =
round(test_errors_all, 4))
print(knn_summary_all)
```

```
##   K Test_Error_Rate
## 1 2           0.5385
## 2 5           0.4904
## 3 9           0.4423
```

(c) How does KNN compare to the logistic regression?

```
lr_test_error = round(test_error_all, 4)
```

```
compare_df = data.frame(Method = c("Logistic Regression (6 predictors)", "KNN
K=2 (6 predictors)", "KNN K=5 (6 predictors)", "KNN K=9 (6 predictors)"),
Test_Error_Rate = c(lr_test_error, test_errors_all))
print(compare_df)
```

```
##                               Method Test_Error_Rate
## 1 Logistic Regression (6 predictors)      0.5865000
## 2                               KNN K=2 (6 predictors)      0.5384615
## 3                               KNN K=5 (6 predictors)      0.4903846
## 4                               KNN K=9 (6 predictors)      0.4423077
```

Comment: Logistic regression tends to be more stable for binary classification on financial data like Weekly, as it makes explicit use of the linear decision boundary assumption. KNN is a non-parametric method and can be sensitive to the choice of K, especially when predictors have noise (as is the case with market return lags). For small K (e.g., K=2), KNN tends to overfit, resulting in higher test error. As K increases, the model smooths out and may perform comparably to logistic regression. Overall, neither method dramatically outperforms random guessing on this dataset, reflecting the weak predictability of weekly returns.

(d) Compute the test error rates for each K, if instead of choosing all predictors we choose only lag 1 and lag 2.

```
pred_cols2 = c("Lag1", "Lag2")
```

```
X_train2 = scale(train[, pred_cols2])
X_test2 = scale(test[, pred_cols2], center = attr(X_train2, "scaled:center"),
scale = attr(X_train2, "scaled:scale"))
set.seed(42)
```

```

test_errors_lag12 = numeric(length(K_vals))

for (i in seq_along(K_vals)) {
  k = K_vals[i]
  pred_k = knn(train = X_train2, test = X_test2, cl = y_train, k = k)
  pred_k_factor = factor(pred_k, levels = c("Down", "Up"))
  cm_k = confusionMatrix(pred_k_factor, y_test_factor, positive = "Up")
  err = 1 - cm_k$overall["Accuracy"]
  test_errors_lag12[i] = err

  cat("KNN with K =", k, "(Lag1 + Lag2 only)\n")
  print(cm_k$table)
  cat("Test Error Rate:", round(err, 4), "\n\n")
}

## KNN with K = 2 (Lag1 + Lag2 only)
##           Reference
## Prediction Down Up
##           Down  25 29
##           Up   18 32
## Test Error Rate: 0.4519
##
## KNN with K = 5 (Lag1 + Lag2 only)
##           Reference
## Prediction Down Up
##           Down  22 32
##           Up   21 29
## Test Error Rate: 0.5096
##
## KNN with K = 9 (Lag1 + Lag2 only)
##           Reference
## Prediction Down Up
##           Down  18 23
##           Up   25 38
## Test Error Rate: 0.4615

```

Summary Table — Lag1 & Lag2 Only

```

knn_summary_lag12 <- data.frame(
  K           = K_vals,
  All_Predictors = round(test_errors_all, 4),
  Lag1_Lag2_Only = round(test_errors_lag12, 4)
)
cat("Test Error Rate Comparison – All Predictors vs Lag1+Lag2:\n")

## Test Error Rate Comparison – All Predictors vs Lag1+Lag2:

print(knn_summary_lag12)

##   K All_Predictors Lag1_Lag2_Only
## 1 2           0.5385           0.4519

```

## 2 5	0.4904	0.5096
## 3 9	0.4423	0.4615

Comment: Using only Lag1 and Lag2 as predictors for KNN can sometimes improve performance compared to using all six predictors, because irrelevant predictors introduce noise in the distance computation used by KNN. If Volume and higher-order lags add noise rather than signal, dimensionality reduction to Lag1 + Lag2 improves the nearest-neighbour search and reduces the test error rate. The optimal K may also differ across the two predictor sets.
